El montículo personalizado (Custom heap): SplHeap

Caso práctico

Carlos quiere usar un montículo para ordenar los pedidos de la tienda, pero sus pedidos son **objetos** con número, fecha de entrega e importe, y el orden que necesita es "primero la fecha más próxima y, a igual fecha, el de mayor importe". Ni `SplMinHeap` ni `SplMaxHeap` saben hacer eso. **Juan** le explica que las dos heredan de una clase **abstracta**, `SplHeap`, y que él puede crear la suya con **su propio criterio**.

1.- SplHeap: una clase abstracta

`SplHeap` es la clase **abstracta** de la que heredan `SplMinHeap` y `SplMaxHeap`. Tiene todo el funcionamiento del montículo (`insert`, `extract`, `top`, `count`, `isEmpty`, el recorrido...) salvo **cómo se comparan dos elementos**, que es un método abstracto:

```
abstract protected function compare(mixed $value1, mixed $value2): int;
```

Debes conocer

La regla de `compare($a, $b)`: debe devolver un número **positivo si `$a` debe estar más arriba** (salir antes) que `$b`, **0** si son equivalentes y **negativo** en otro caso. Con el operador `<=>`:

- `return $a <=> $b;` → sale primero el **mayor** (como `SplMaxHeap`),
- `return $b <=> $a;` → sale primero el **menor** (como `SplMinHeap`).

Si te equivocas de orden, el montículo funciona... pero al revés. Pon un punto de ruptura dentro de `compare()` para ver qué compara.

2.- Crear un montículo propio

```php
<?php
final class Pedido
{
    public function __construct(
        public readonly int $numero,
        public readonly string $entrega,     // "Y-m-d"
        public readonly float $importe,
    ) {
    }
}

final class ColaPedidos extends SplHeap
{
    protected function compare(mixed $a, mixed $b): int
    {
        // 1.º la fecha MÁS PRÓXIMA ($b delante); 2.º el importe MAYOR ($a delante)
        return [$b->entrega, $a->importe] <=> [$a->entrega, $b->importe];
    }
}
$pedidos = new ColaPedidos();
$pedidos->insert(new Pedido(1001, '2026-10-20', 45.90));
$pedidos->insert(new Pedido(1002, '2026-10-18', 12.50));
$pedidos->insert(new Pedido(1004, '2026-10-18', 89.99));
echo $pedidos->top()->numero;        // 1004: el 18/10 y de más importe
```

Criterios con varios niveles. Comparar **arrays** con `<=>` compara elemento a elemento, así que `[criterio1, criterio2, ...]` resuelve los desempates. En cada posición, pon `$a` a la izquierda si quieres el **mayor** primero y `$b` si quieres el **menor**.

Otras formas:

- Heredar de `SplMinHeap` o `SplMaxHeap` y redefinir `compare()` (se mantiene el sentido, pero cambia el criterio). Por ejemplo, para ordenar nombres en español sin que las mayúsculas y las tildes estropeen el orden, se compara con un `Collator('es_ES')` de la extensión intl.
- Pasar la función de comparación al construir el montículo (una *closure*), para tener una sola clase con cualquier criterio:

```php
<?php
final class HeapConComparador extends SplHeap
{
    public function __construct(private Closure $comparador)
    {
    }

    protected function compare(mixed $a, mixed $b): int
    {
        return ($this->comparador)($a, $b);
    }
}
$masBarato = new HeapConComparador(fn($a, $b) => $b['precio'] <=> $a['precio']);
```

3.- Montículos corruptos

Si `compare()` **lanza una excepción** mientras se inserta o extrae un elemento (por ejemplo, porque un pedido no tiene fecha), el montículo puede quedar a medio reordenar. PHP lo marca como **corrupto**:

- `isCorrupted()` devuelve `true`,
- cualquier operación posterior lanza `RuntimeException: Heap is corrupted, heap properties are no longer ensured`,
- `recoverFromCorruption()` quita la marca, pero **el orden ya no está garantizado** (y el elemento erróneo puede haberse quedado dentro).

Recomendación

Valida los datos **antes** de que lleguen a `compare()`: redefine `insert()` para comprobar el tipo o los campos y lanzar la excepción allí. Así el error es claro y el montículo nunca se corrompe.

```
<?php
final class ColaTareasSegura extends SplHeap
{
    public function insert(mixed $valor): true
    {
        if (!$valor instanceof Tarea) {
            throw new TypeError('Solo se admiten objetos Tarea, no ' .
get_debug_type($valor));
        }
        return parent::insert($valor);
    }

    protected function compare(mixed $a, mixed $b): int
    {
        return $a->prioridad->value <=> $b->prioridad->value;
    }
}
```

4.- Aplicaciones típicas

Problema	Criterio de `compare()`
Pedidos, tareas, incidencias con varios criterios	`[prioridad, plazo, duración]` en el orden adecuado
Rankings con una puntuación calculada	La puntuación (por ejemplo, estrellas ponderadas por el número de votos)
Ordenar textos según el idioma	`Collator::compare()`
Planificar ascensores, robots, rutas	La urgencia y la distancia
Un "montículo con tipo"	El criterio, más una comprobación en `insert()`

5.- Ejercicio resuelto: el gestor de tareas del equipo

Ejercicio resuelto

El equipo de desarrollo trabaja primero en las tareas de **mayor prioridad**; a igual prioridad, en la que **vence antes**; y si también coinciden, en la **más corta**.

```
<?php
enum Prioridad: int
```

```
{
    case Baja = 1;
    case Media = 2;
    case Alta = 3;
}
final class Tarea
{
    public function __construct(
        public readonly string $titulo,
        public readonly Prioridad $prioridad,
        public readonly DateTimeImmutable $plazo,
        public readonly int $horas,
    ) {
    }
}
final class ColaTareas extends SplHeap
{
    protected function compare(mixed $a, mixed $b): int
    {
        return [$a->prioridad->value, $b->plazo, $b->horas]    // prioridad MAYOR, plazo
MENOR, horas MENOS
            <=> [$b->prioridad->value, $a->plazo, $a->horas];
    }
}
$tareas = new ColaTareas();
$tareas->insert(new Tarea('Actualizar dependencias', Prioridad::Media, new
DateTimeImmutable('2026-10-30'), 2));
$tareas->insert(new Tarea('Arreglar el pago con Bizum', Prioridad::Alta, new
DateTimeImmutable('2026-10-17'), 5));
$tareas->insert(new Tarea('Error en el carrito', Prioridad::Alta, new
DateTimeImmutable('2026-10-17'), 3));
$tareas->insert(new Tarea('Copia de seguridad', Prioridad::Alta, new
DateTimeImmutable('2026-10-20'), 1));
while (!$tareas->isEmpty()) {
    echo $tareas->extract()->titulo, PHP_EOL;
}
// Error en el carrito / Arreglar el pago con Bizum / Copia de seguridad / Actualizar
dependencias
```

Observa que los objetos `DateTimeImmutable` y los casos del enum (a través de su `value`) se pueden comparar directamente dentro del array.

Autoevaluación

- ¿Se puede hacer `new SplHeap()`? **No: es abstracta. Hay que heredar e implementar `compare()`.**
- En un `SplHeap` propio, ¿qué sale primero si `compare` es `return $a <=> $b;`? **El mayor.**
- ¿Qué ocurre si `compare()` lanza una excepción durante una inserción? **El montículo queda corrupto y las siguientes operaciones lanzan `RuntimeException`.**
- ¿Cómo evitarías que se corrompa? **Validando los datos en un `insert()` redefinido, antes de llamar a `parent::insert()`.**

6.- Ejercicios

Fichero	Contenido
`Ejercicios CustomHeap/e01-SplHeap.php`	Montículo de objetos, heredar de `SplMinHeap` con `Collator`, comparador como parámetro, montículos corruptos
`Ejercicios CustomHeap/e02-SplHeap.php`	Aplicaciones: gestor de tareas, ranking ponderado (top K), montículo con tipo
`Ejercicios CustomHeap/problemas/`	`e01` y `e02`: problemas y soluciones
